



LeetCode 118: Pascal's Triangle

1. Problem Title & Link

Title: LeetCode 118: Pascal's Triangle

Link: <https://leetcode.com/problems/pascals-triangle/>

2. Problem Statement (Short Summary)

Given an integer numRows, generate the first numRows of Pascal's Triangle.

In Pascal's Triangle:

- First and last element of every row is 1
- Every middle element is the sum of the two elements directly above it

Return the triangle as a 2D list.

Example

Input:

numRows = 5

Output:

```
[  
  [1],  
  [1,1],  
  [1,2,1],  
  [1,3,3,1],  
  [1,4,6,4,1]  
]
```

3. Examples (Input → Output)

Example 1

Input:

numRows = 1

Output:

```
[[1]]
```

Example 2

Input:

numRows = 2

Output:

```
[  
  [1],
```



```
[1,1]
]
```

Example 3

Input:

numRows = 5

Output:

```
[
  [1],
  [1,1],
  [1,2,1],
  [1,3,3,1],
  [1,4,6,4,1]
]
```

Example 4

Input:

numRows = 6

Output:

```
[
  [1],
  [1,1],
  [1,2,1],
  [1,3,3,1],
  [1,4,6,4,1],
  [1,5,10,10,5,1]
]
```

4. Constraints $1 \leq \text{numRows} \leq 30$

Important:

Need to generate all rows.

Return a 2D list.

5. Core Concept (Pattern / Topic)**Dynamic Programming**

Secondary Topics:



- 2D Arrays
- Pattern Generation
- Simulation

6. Thought Process (Step-by-Step Explanation)

Understanding Pascal's Triangle

Let's write the first few rows.

Row 0 [1]

Row 1 [1 1]

Row 2 [1 2 1]

Row 3 [1 3 3 1]

Row 4 [1 4 6 4 1]

Observe:

Every row starts with 1.

Every row ends with 1.

How Are Middle Values Generated?

Look at Row 3:

Row 2

[1 2 1]

Build Row 3:

1

$1+2 = 3$

$2+1 = 3$

1

Result:

[1,3,3,1]

Another Example



Build Row 4 from Row 3

Previous Row

[1,3,3,1]

Calculate:

1

$1+3 = 4$

$3+3 = 6$

$3+1 = 4$

1

Result:

[1,4,6,4,1]

Dynamic Programming Observation

Each element depends on values already computed in the previous row.

Specifically:

current[j]

=

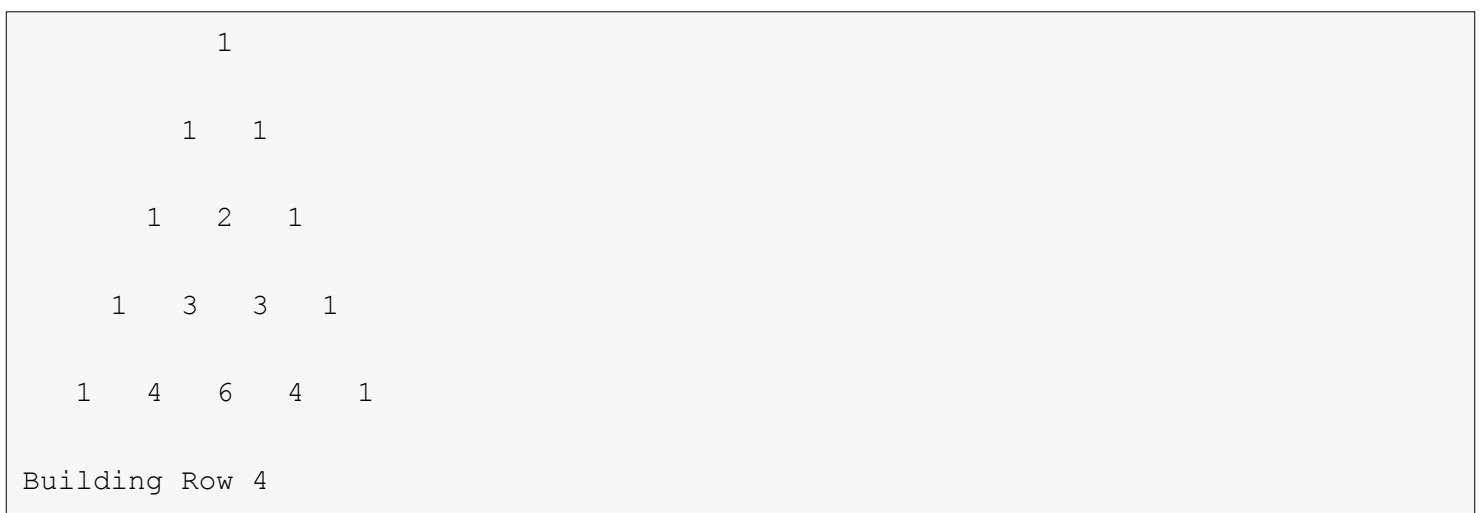
previous[j-1]

+

previous[j]

This is why Pascal's Triangle is considered a Dynamic Programming problem.

7. Visual / Intuition Diagram (ASCII Diagram)





Previous Row

1 3 3 1

↓ ↓

1+3 = 4

↓ ↓

3+3 = 6

↓ ↓

3+1 = 4

Result:

1 4 6 4 1

8. Pseudocode (Language Independent)

```
triangle = []

for row from 0 to numRows-1

    create current row

    add 1 at beginning

    for each middle position

        value =
            previousRow[col-1]
            +
            previousRow[col]

        add value

    if row > 0
        add ending 1

    add current row to triangle

return triangle
```



9. Code Implementation

Python

```
class Solution:
    def generate(self, numRows: int):

        triangle = []

        for row in range(numRows):

            current = [1]

            if row > 0:

                previous = triangle[row - 1]

                for col in range(1, row):

                    current.append(
                        previous[col - 1] + previous[col]
                    )

                current.append(1)

            triangle.append(current)

        return triangle
```

Java

```
import java.util.*;

class Solution {

    public List<List<Integer>> generate(int numRows) {

        List<List<Integer>> triangle = new ArrayList<>();

        for(int row = 0; row < numRows; row++) {

            List<Integer> current = new ArrayList<>();

            current.add(1);
```



```
        if(row > 0) {

            List<Integer> previous =
                triangle.get(row - 1);

            for(int col = 1; col < row; col++) {

                current.add(
                    previous.get(col - 1)
                    +
                    previous.get(col)
                );
            }

            current.add(1);

            triangle.add(current);
        }

        return triangle;
    }
}
```

10. Time & Space Complexity

Time Complexity

$O(\text{numRows}^2)$

Reason:

Row 1 → 1 element

Row 2 → 2 elements

Row 3 → 3 elements

...

Total:

$1 + 2 + 3 + \dots + n$

$= O(n^2)$

Space Complexity

$O(\text{numRows}^2)$



Need to store the entire triangle.

11. Common Mistakes / Edge Cases

Mistake 1

Forgetting first and last element.

Every row must begin and end with:

1

Mistake 2

Using current row instead of previous row.

Wrong:

```
current[col-1]
```

+

```
current[col]
```

Need:

```
previous[col-1]
```

+

```
previous[col]
```

Mistake 3

Looping one position too far.

Careful:

```
for col in range(1, row):
```

not

```
range(1, row+1)
```

Edge Cases

numRows = 1

Output:

```
[[1]]
```

numRows = 2

Output:

```
[  
  [1],  
  [1,1]  
]
```



12. Detailed Dry Run

Input:

numRows = 5

Row 0

[1]

Triangle:

[[1]]

Row 1

[1,1]

Triangle:

```
[  
  [1],  
  [1,1]  
]
```

Row 2

Using:

[1,1]

Middle:

$1+1=2$

Result:

[1,2,1]

Row 3

Using:

[1,2,1]

Middle:

$1+2=3$

$2+1=3$

Result:

[1,3,3,1]

Row 4



Using:

[1,3,3,1]

Middle:

$1+3=4$

$3+3=6$

$3+1=4$

Result:

[1,4,6,4,1]

Final Answer:

```
[  
  [1],  
  [1,1],  
  [1,2,1],  
  [1,3,3,1],  
  [1,4,6,4,1]  
]
```

13. Common Use Cases (Real-Life / Interview)

Combinations

Pascal's Triangle directly represents:

nCr

values.

Example:

$5C2 = 10$

appears in Pascal's Triangle.

Probability

Used in:

Binomial Distribution

Dynamic Programming

One of the simplest DP examples.

Combinatorics

Used heavily in mathematics.



14. Common Traps (Important!)

Trap 1

Not recognizing dependency on previous row.

Trap 2

Trying to compute factorials.

Not needed.

Trap 3

Forgetting:

Row length = row number + 1

Example:

Row 4

contains 5 elements.

15. Builds To (Related LeetCode Problems)

LC 118 - Pascal's Triangle

↓

LC 119 - Pascal's Triangle II

↓

LC 509 - Fibonacci Number

↓

LC 70 - Climbing Stairs

↓

LC 62 - Unique Paths

↓

LC 64 - Minimum Path Sum

These build Dynamic Programming fundamentals.

16. Alternate Approaches + Comparison

Approach	Time	Space	Interview Rating
Simulation	$O(n^2)$	$O(n^2)$	Best
nCr Formula	$O(n^2)$	$O(n^2)$	Advanced Math



Recursive Generation	$O(n^2)$	$O(n^2)$	Less Preferred
----------------------	----------	----------	----------------

Approach 1: Simulation (Chosen)

Generate rows one by one.

Most readable.

Approach 2: Combination Formula

Each value can be calculated as:

$${}^nC_r = \frac{n!}{r!(n-r)!}$$

More mathematical but less intuitive.

17. Why This Solution Works (Short Intuition)

Every row is built using the row immediately above it.

Each middle value is the sum of two adjacent values from the previous row:

previous[col-1]

+

previous[col]

By repeatedly applying this rule, the entire triangle is generated correctly.

18. Variations / Follow-Up Questions

Follow-Up 1

Return only a specific row.

Related:

LC 119 - Pascal's Triangle II

Follow-Up 2

Find the sum of a row.

Observation:

Row 0 sum = 1

Row 1 sum = 2

Row 2 sum = 4

Row 3 sum = 8

Pattern:

2^{row}

Follow-Up 3

Print Pascal's Triangle in pyramid format.



Example:

```
1
1 1
1 2 1
1 3 3 1
```

Follow-Up 4

Find:

nCr

using Pascal's Triangle instead of factorials.

Interview Takeaway

Pascal's Triangle is often the **first true Dynamic Programming problem** students encounter.

The key insight is:

Current Value

=

Two Values Above It

This teaches the fundamental DP idea:

Current Answer

Depends On

Previously Computed Answers

which appears repeatedly in interview problems like Climbing Stairs, Unique Paths, Coin Change, and House Robber.